HDOC Day-55

****

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k <= size. Pass the array to a function named window_maximum() using its address. Use pointers to pass the array between the calling function and window_maximum(). The function window_maximum() must:

- Consider every contiguous subarray of size k as a sliding window.
- Find the maximum element in each window while moving from left to right.
- Display the maximum elements in the order of their corresponding windows.
- Separate the maximum elements by spaces.
- Display -1 if k is invalid.
- Use a nested-loop approach to solve the problem.

1. Problem Understanding

The program reads the size, array elements, and k. It passes the array address to window_maximum(). The function treats every contiguous subarray of size k as a sliding window, finds the maximum element in each window using nested loops, and displays the maxima from left to right separated by spaces. If k is invalid, it displays -1.

2. Standard Test Cases

Test Case	Size (n)	Array Input	k	Expected Output
1	8	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	5	1 2 3 4 5	2	2 3 4 5
3	3	1 2 3	4	-1
4	5	5 4 3 2 1	3	5 4 3
5	4	2 8 1 6	1	2 8 1 6

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare a variable-length integer array arr[n].
4. Read the array elements.
5. Read k.
6. Pass arr, n and k to window_maximum() using the array address.
7. Stop.

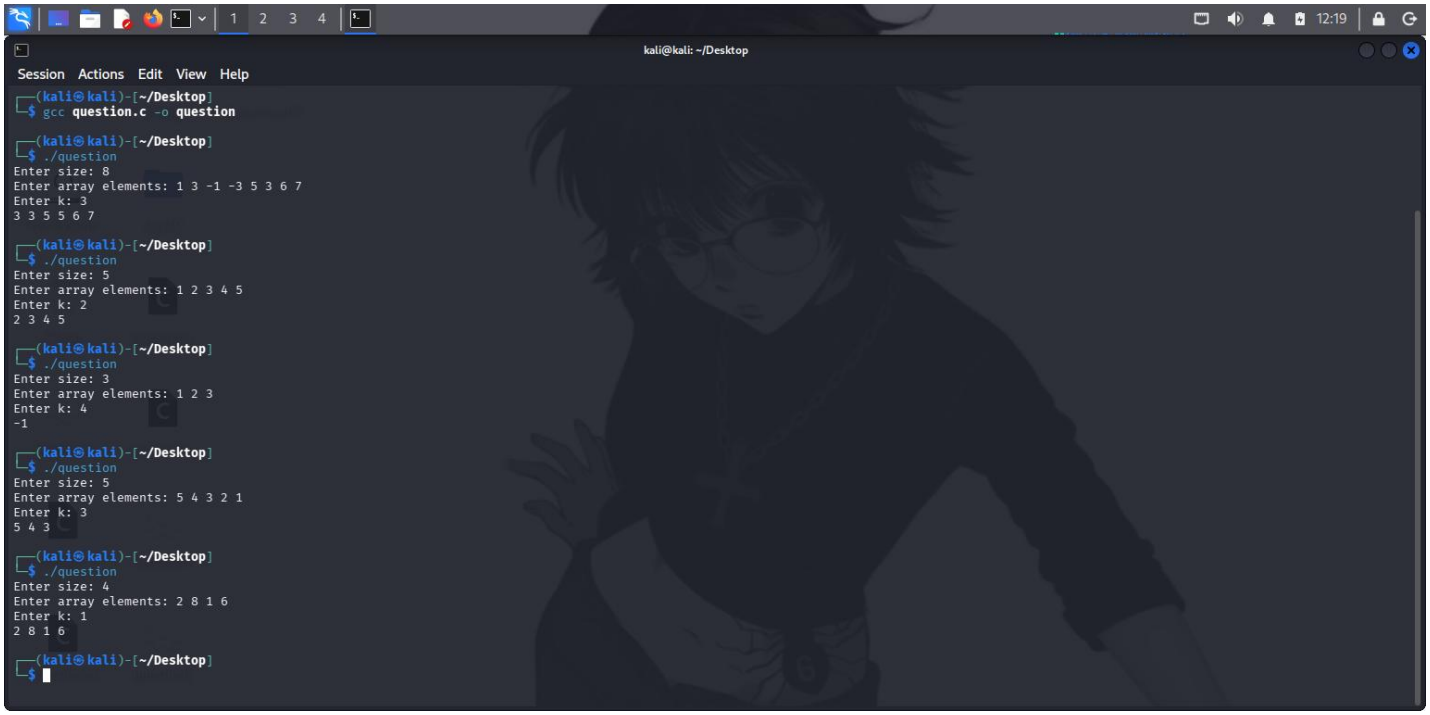
Function: window_maximum()

1. If k <= 0 or k > n, display -1 and return.
2. For each possible window starting at index i from 0 to n-k, set max = *(arr + i).
3. Use an inner loop from j = 1 to k-1.
4. Compare *(arr + i + j) with max and update max when a greater element is found.
5. Display max for the current window.
6. Separate successive maximum values by spaces.

4. C Program

```
#include <stdio.h>
void window_maximum(int *arr, int n, int k)
{
   int i, j, max;
   if(k <= 0 || k > n)
   {
      printf("-1\n");
      return;
   }
   for(i = 0; i <= n - k; i++)
   {
      max = *(arr + i);
      for(j = 1; j < k; j++)
      {
         if(*(arr + i + j) > max)
         {
            max = *(arr + i + j);
         }
      }
      printf("%d", max);
      if(i < n - k)
         printf(" ");
   }
   printf("\n");
}
int main()
{
   int n, i, k;
   printf("Enter size: ");
   scanf("%d", &n);
   int arr[n];
   printf("Enter array elements: ");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }
   printf("Enter k: ");
   scanf("%d", &k);
   window_maximum(arr, n, k);
   return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	k	Actual Result	Status
1	8	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7	Pass
2	5	1 2 3 4 5	2	2 3 4 5	Pass
3	3	1 2 3	4	-1	Pass
4	5	5 4 3 2 1	3	5 4 3	Pass
5	4	2 8 1 6	1	2 8 1 6	Pass